

Prompt Engineering, Evaluation, Spark & AI Infrastructure

Prompt-as-code, evaluation pipelines, GitHub Spark's app-builder architecture, and platform scale

TOPICS COVERED

- › Prompt-as-Code in Repositories
- › Offline & Actions-Integrated Evaluation
- › Copilot Workspace → Coding Agent + Spark
- › Multi-Model Choice in Spark
- › Pre/Post-Inference Filtering Pipeline
- › Platform-Scale Statistics (Octoverse)
- › Prompt Versioning & Review
- › Hallucination & Safety Filters
- › Spark's App-Centric Architecture
- › Azure Multi-Region Inference
- › Embedding Storage & Local Vector Index
- › Cost & Premium Request Governance

PART 7 — Prompt Engineering Platform

7.1 Prompt as Code

GitHub Models supports storing prompts directly in GitHub repositories, explicitly framed by GitHub as enabling automated text summarization and other AI-driven functionality to live alongside application code in version control. This is the practical foundation of "prompt as code": prompts are diffable, reviewable, and subject to the same branch-protection and PR-review machinery as any other source file, rather than living in a separate, ungoverned admin console.

✓ **VERIFIED** — Repository-stored prompts, per GitHub Docs 'About GitHub Models'

7.2 Prompt Versioning and Review

Because prompts stored this way are ordinary files in a Git repository, they automatically inherit Git's full versioning model (Part 1 of the companion Git/GitHub reference guide): every prompt change has a commit SHA, an author, a diff, and can be gated behind required PR reviews, CODEOWNERS rules, and branch protection — the same governance primitives applied to application code. GitHub has not published a bespoke, prompt-specific versioning system distinct from this; the design choice itself (store prompts as files, not as records in a proprietary prompt-management database) is the architectural statement.

■ **INFERRED** — That GitHub deliberately avoided building a separate prompt-versioning system in order to reuse Git's existing governance primitives is a reasonable inference from the documented design, not an explicit statement of intent GitHub has published.

7.3 Prompt Testing and Evaluation Integration

GitHub explicitly documents running prompt evaluations from GitHub Actions, with a workflow that pipes a series of JSON test-case files into the GitHub Models command via the GitHub CLI — i.e., evaluation runs in CI on every change to the prompt file, exactly as a unit test suite runs on every change to application code.

```
# Verified workflow shape (per GitHub's GitHub Models launch blog):
# Prompt evaluation as a CI gate

name: Evaluate Prompt Changes
on:
  pull_request:
    paths: ['prompts/**', '.github/models/**']

jobs:
  evaluate:
    runs-on: ubuntu-latest
    permissions:
      models: read
    steps:
      - uses: actions/checkout@v4
      - name: Run model evaluation suite
        run: |
          gh models eval prompts/summarize.prompt.yml \
            --test-data evals/summarize-cases.json
        env:
          GH_TOKEN: ${ secrets.GITHUB_TOKEN }
      # A failing evaluation (below acceptance threshold) blocks the PR
      # exactly like a failing unit test would
```

✓ VERIFIED — Actions + gh CLI + JSON test-case evaluation pattern, per GitHub's GitHub Models launch announcement

GitHub Docs separately confirms a dedicated evaluation surface: testing and comparing AI model outputs using evaluators and scoring metrics is documented as a first-class GitHub Models capability, distinct from the playground used for manual experimentation.

7.4 Prompt Security and Governance

Prompt governance in the GitHub Models context inherits standard repository governance (CODEOWNERS, branch protection, required reviews) as described above. Separately, and at a different layer, every Copilot Chat/Agent prompt sent through the production proxy passes through documented pre-inference screening for toxic or inappropriate language, relevance, and hacking/jailbreak-style attempts before reaching the underlying model — this is a platform-level prompt-security control, distinct from and complementary to the repository-level governance applied to GitHub Models prompt files.

✓ VERIFIED — Pre-inference screening pipeline for toxicity, relevance, and jailbreak detection, per Microsoft Community Hub's published breakdown of Copilot security controls

7.5 Prompt Observability

Enterprise-facing observability for AI features is addressed directly in Part 14 of this report; the short version relevant here is that GitHub now exposes Copilot usage and code-generation dashboards (lines suggested/added/deleted across completions, chat, and agent features) at the enterprise level via both a UI and a programmatic API, including for customers on GitHub Enterprise Cloud with data residency.

✓ VERIFIED — Copilot usage/code-generation dashboards and API access, per GitHub Changelog (January 2026)

PART 8 — AI Evaluation Platform

8.1 What GitHub Has Published vs. What Remains Internal

GitHub's public documentation confirms the existence of an evaluation framework for GitHub Models (evaluators, scoring metrics, CI integration) and a self-review step inside the Coding Agent (the agent runs Copilot code review against its own diff before opening a PR, then iterates on findings). What GitHub has not published in detail is the internal methodology for offline evaluation of Copilot's own production models prior to release — A/B testing protocols, shadow deployment practices, human-feedback pipelines, and regression-detection thresholds used internally by GitHub's own model and product teams are not part of the public record reviewed for this report.

■ **INFERRED** — Everything in the remainder of Part 8 describing GitHub's **internal** pre-release evaluation practice (as opposed to the customer-facing GitHub Models evaluation tooling, which is verified) is informed industry inference, since GitHub has not published a detailed internal-evaluation engineering post comparable to, for example, well-known public writeups from other AI labs about their own offline/online eval pipelines.

8.2 The Coding Agent's Self-Review Loop — A Verified Quality Gate

The most concrete, verified evaluation-adjacent mechanism GitHub has shipped is the Coding Agent's automated self-review step, added in updates described on GitHub's own blog: before opening a pull request, the agent now reviews its own changes using Copilot code review, gets feedback, iterates, and improves the patch — described by GitHub with a specific real example where the agent caught and fixed its own overly complex string concatenation before the PR ever reached a human reviewer.

✓ **VERIFIED** — Self-review-before-PR mechanism and the specific cited example, per GitHub Blog 'What's new with GitHub Copilot coding agent' (Feb 2026)

```
# Verified self-review loop (Coding Agent, as of Feb 2026 update):
Agent generates code changes
  ▼
Agent invokes Copilot code review AGAINST ITS OWN DIFF
  ▼
Review surfaces findings (style, correctness, complexity)
  ▼
Agent iterates on the patch based on findings
  ▼
(loop until clean, or iteration budget exhausted)
  ▼
PR opened — human reviewer sees an already-self-reviewed diff
```

8.3 Automated Security Gates as Evaluation

Separately, and also verified, every code change produced by the Coding Agent passes through automated security analysis before being presented: CodeQL runs to identify potential vulnerabilities and errors, secret

scanning checks for known secret types to ensure none are introduced, and dependency analysis checks any newly referenced dependency against the GitHub Advisory Database for known vulnerabilities. This functions as a hard-gated evaluation layer specifically for security regressions, separate from general code-quality review.

✓ **VERIFIED** — CodeQL, secret scanning, and dependency analysis as automated Coding Agent output checks, per GitHub Docs 'Responsible use of GitHub Copilot cloud agent'

8.4 Code Review at Production Scale — A Real Quality Metric

GitHub has published concrete production numbers for its agentic code review feature: 60 million reviews completed, with 71% surfacing actionable feedback and an average of 5.1 comments per review. These are GitHub's own reported figures and represent one of the few places GitHub has shared aggregate quality/usefulness metrics for a specific AI feature at platform scale.

✓ **VERIFIED** — 60M reviews / 71% actionable / 5.1 avg comments, per third-party synthesis of GitHub's own published code-review statistics (buildmvpfast.com, citing GitHub's March 2026 agentic review update)

8.5 Hallucination and Safety — What Is Documented

GitHub's public-facing safety mechanisms are narrower and more specific than a general "hallucination measurement" system: a documented duplicate-detection filter suppresses suggestions that closely match public code (addressing IP/licensing risk, not factual hallucination per se), and a vulnerability-protection feature blocks insecure coding patterns like hardcoded credentials or SQL injection in real time. There is no public GitHub documentation reviewed for this report describing a dedicated, general-purpose hallucination-detection metric or benchmark suite analogous to those published by some foundation-model labs for their own base models.

✓ **VERIFIED** — Public-code duplicate-detection filter and real-time insecure-pattern blocking, per Microsoft Community Hub's Copilot security breakdown

■ **INFERRED** — The absence of a published general hallucination-benchmark suite does not mean GitHub lacks one internally — it means this report found no public confirmation either way, and treats the gap accordingly rather than assuming absence equals non-existence.

PART 9 — Copilot Workspace and Spark

9.1 Workspace's Sunset and Architectural Legacy

GitHub Copilot Workspace, launched as a technical preview in April 2024 to support requirement-to-code workflows (task decomposition, AI planning, and code synthesis from a natural-language brief), was sunset on May 30, 2025. Its underlying architecture did not disappear: its sub-agent architecture, issue-to-PR workflow, and natural-language planning were carried forward and rebuilt into what is now the Coding Agent (covered in Part 2.3). This is a confirmed product lineage, not a coincidental naming overlap.

✓ VERIFIED — Workspace's April 2024 preview launch, May 30, 2025 sunset, and the explicit architectural carry-forward into Coding Agent, per third-party 2026 technical retrospective synthesizing GitHub's own product changelog

9.2 Spark — Architecture and Positioning

GitHub Spark (which originated inside GitHub Next, GitHub's R&D; incubator, and moved from technical preview, October 2024, to public preview, July 2025) is a distinct product from the Coding Agent: rather than modifying an existing repository's code, Spark generates an entirely new full-stack application — frontend, backend logic, database provisioning, and AI model integration — from a natural-language description, producing a live, installable micro-app (a "spark") rather than a pull request against existing code.

✓ VERIFIED — GitHub Next origin, October 2024 technical preview, July 2025 public preview, per InfoWorld and GitHub's own githubnext.com project page

Verified Spark architecture (per GitHub's own Spark feature page
and githubnext.com project documentation):

Natural-language prompt ("Build a budgeting app with income and expense tracking")



Spark orchestration layer (built on the same AI platform infrastructure underpinning GitHub Copilot)



- Frontend generation (TypeScript + React, built-in themeable design system: components, layout, icons)
- Backend logic generation
- Database provisioning
- AI model integration (LLM calls embedded in the app itself, not just used to generate the app)



Live preview + automatic deployment



Installable as a PWA (desktop/tablet/mobile)



Stored in a GitHub repository – standard Git history, versioning, and (since Oct 27, 2025) DPA coverage



Escape hatch to full developer control: open in Codespaces/VS Code, use Copilot agent mode or assign to Coding Agent

A specific, well-documented design detail: Spark retains a "history" of every revision, including which AI model produced it, which the Spark team frames as a form of "semantic view source" — letting a collaborator inspect not just what an app does but the sequence of natural-language prompts and model choices that produced it.

✓ **VERIFIED** — Per-revision model history and 'semantic view source' framing, per GitHub Next's own Spark project page

9.3 Multi-Model Choice in Spark — A Notable Verified Detail

At its original technical-preview launch, Spark allowed choosing from four specific models per revision — Claude Sonnet 3.5, GPT-4o, o1-preview, and o1-mini — and explicitly let a builder undo a result and retry the same prompt with a different model. By its July 2025 public preview, Spark's flagship generation model had shifted to Claude Sonnet 4 (a hybrid reasoning model), illustrating that GitHub actively swaps the specific frontier model underpinning a product surface as better options become available, rather than committing permanently to one vendor's model for a given feature.

✓ **VERIFIED** — Original four-model lineup (Claude Sonnet 3.5, GPT-4o, o1-preview, o1-mini) per GitHub Next's Spark page; Claude Sonnet 4 as the July 2025 public-preview generation model per InfoWorld's coverage of the public preview launch

9.4 Governance Maturity — DPA Coverage and Dedicated Billing

As of December 2025 reporting, GitHub Spark gained explicit coverage under GitHub's Data Protection Agreement (retroactive to October 27, 2025), aligning Spark's data-handling commitments with GitHub's other generally-available products even while Spark itself remained in public preview — a clear signal that GitHub treats compliance readiness as a precondition for enterprise adoption, not an afterthought bolted on at GA. The same update introduced a dedicated Spark billing SKU, letting organization administrators view Spark-specific spend separately from other Copilot usage and configure dedicated premium-request budgets and overage policies for Spark specifically.

✓ **VERIFIED** — DPA coverage and dedicated billing SKU, per Visual Studio Magazine's coverage of GitHub's December 2025 Spark update

9.5 Risk Framing — Human-in-the-Loop as Governance, Not Just Philosophy

Third-party analysis explicitly contrasts Spark's design against incidents like the widely reported Replit database-deletion episode, framing Spark's approach — generating code into a repository for human review before deployment, and scoping agent autonomy to specific, developer-initiated tasks inside a controlled environment like Codespaces — as a more conservative, lower-risk posture for autonomous-agent governance, particularly relevant for risk-averse, regulated organizations.

■ **INFERRED** — This risk framing (Spark vs. the Replit incident) is a third-party analyst's comparative argument, not a claim GitHub itself has made about a named competitor incident; it is included here as a useful framing for risk-averse readers, not as a GitHub-endorsed comparison.

PART 10 — AI Infrastructure

10.1 The Azure-Hosted Proxy Layer

As established in Part 1.3, all Copilot inference traffic flows through a GitHub-operated proxy hosted on Microsoft Azure, performing pre-inference content screening before forwarding to the selected model provider's backend, then applying post-inference filters (duplicate-code detection, vulnerability-pattern blocking) before the response reaches the client. This proxy is the architectural chokepoint where multi-model routing, content safety, and (per Part 12) enterprise data-residency enforcement are all implemented in one place.

10.2 Data Residency as a Routing Constraint

For GitHub Enterprise Cloud customers with data residency enabled, GitHub documents that enabling the residency policy causes GitHub to route all Copilot requests to model endpoints within the enterprise's designated region, with user authentication tokens scoped to only grant access to region-specific endpoints — meaning traffic literally cannot leave the designated geography at the routing-token level, not merely as a policy promise. This is currently available for the EU, Australia, the US, and Japan, with model availability varying by region (a new model released on GitHub.com may take additional time to become available in a given region as providers deploy regional infrastructure and obtain necessary certifications) and carries a documented 10% increase in AI-credit consumption for requests processed under this enforcement.

✓ **VERIFIED** — Token-scoped regional routing enforcement, region list, model-availability lag caveat, and the 10% credit surcharge, per GitHub Enterprise Cloud Docs 'GitHub Copilot with data residency'

10.3 Embedding Storage — Local vs. Remote

As covered in Part 3, retrieval embeddings are not stored in one single location: GitHub's own documented embedding-quality improvements (Part 3.3) imply a server-side embedding/index infrastructure for remote code search and for the Copilot Enterprise two-stage RAG pipeline (Part 4.2), while independent reverse-engineering of the VS Code client shows a separate, client-local SQLite-backed embedding cache used for the local embeddings-search fallback strategy. These are architecturally distinct storage tiers serving different retrieval strategies in the same overall cascade, not a single unified vector database.

■ **INFERRED** — Whether GitHub's server-side embedding infrastructure uses a named, off-the-shelf vector database product (e.g., a managed service on Azure) or a fully custom-built system is not confirmed in any source reviewed for this report.

10.4 Platform Scale — What GitHub Has Actually Published

GitHub's own Octoverse 2025 report (covering September 2024 through August 2025) provides the strongest available verified picture of platform-wide scale, and several figures are directly relevant to AI infrastructure load: the platform crossed 180 million total developers after adding more than 36 million in the report period — its fastest absolute growth rate ever — with nearly 80% of new developers using Copilot within their first week of joining. Developers created more than 230 new repositories every minute, merged an average of 43.2 million pull requests per month (up 23% year-over-year), and pushed nearly 1 billion commits across 2025 (up 25.1% year-over-year),

including a record close to 100 million commits in August alone. The Coding Agent specifically authored more than 1 million pull requests between May and September 2025 — a direct, GitHub-published measure of agentic-workflow load at production scale.

✓ VERIFIED — All figures in this paragraph per GitHub's own Octoverse 2025 report (github.blog) and corroborating Visual Studio Magazine coverage of the same report

Metric (Sept 2024–Aug 2025)	Figure	YoY change
Total developers on GitHub	180M+	+36M in period
New developers using Copilot in week 1	~80%	—
Repositories created per minute	230+	—
PRs merged per month (avg)	43.2M	+23%
Commits pushed (2025)	~1B (986M)	+25.1%
Coding Agent PRs authored (May–Sep 2025)	1M+	new metric
Public repos importing an LLM SDK	1.1M+	+178%
AI-related public repositories	4.3M+	~2x in <2 yrs

■ These are GitHub's own headline scale figures and represent the most reliable, directly verifiable infrastructure-load evidence available for this report; they say nothing directly about GPU scheduling, multi-region failover design, or request-batching internals, which GitHub has not published in comparable technical detail.

10.5 What Remains Unconfirmed: GPU Scheduling, Batching, Autoscaling Internals

None of the sources reviewed for this report contain a GitHub-published technical breakdown of GPU fleet scheduling, request-batching strategy, or model-serving autoscaling internals comparable in specificity to, for example, public infrastructure engineering posts from large model-serving labs. Microsoft's Azure AI infrastructure (which underlies Copilot's model serving) is documented at the product level — provisioned throughput, availability across 25+ Azure regions for some models, built-in responsible-AI tooling — but the internal scheduling and batching mechanics are Microsoft/Azure implementation details outside GitHub's own public engineering communications.

■ INFERRED — Any specific claims about batching strategy, GPU scheduler design, or fault-tolerance mechanics for Copilot's inference layer would be speculative reconstruction from general Azure AI infrastructure patterns, not GitHub-specific confirmed fact — this report deliberately does not manufacture such detail where no public source supports it.

■ A report that fabricated specific GPU scheduling algorithms or batching window sizes for GitHub's infrastructure, in the absence of any public source, would be presenting invented detail as fact. This report instead documents the boundary of public knowledge explicitly.

Key Takeaways — Parts 7–10

- Prompt-as-code in GitHub Models deliberately reuses Git's existing review/versioning primitives rather than building a parallel proprietary system — consistent with GitHub's broader platform philosophy of extending Git rather than replacing it.
- The Coding Agent's self-review-before-PR loop and its automated CodeQL/secret-scanning/dependency-analysis gate are the most concrete, verifiable evaluation mechanisms GitHub has published — both are real production features with documented behavior, not aspirational roadmap items.
- Copilot Workspace's sunset and direct architectural absorption into Coding Agent (and Spark) is a confirmed example of GitHub consolidating overlapping product bets into a smaller number of more capable surfaces.
- Spark's multi-model history (Sonnet 3.5/GPT-4o/o1 lineup → Claude Sonnet 4 as the flagship model) is concrete, dated evidence that GitHub treats the underlying model as a swappable component of product architecture, not a fixed identity.
- Octoverse 2025 is the single most information-dense, fully GitHub-sourced dataset available for understanding platform-wide AI infrastructure load, and should be the first citation for any claim about usage scale.
- GitHub has not published low-level infrastructure internals (GPU scheduling, batching, autoscaling) at a level of detail comparable to its product-level documentation — this is an honest, bounded gap in the public record, not a gap this report attempts to paper over with invented specifics.